Claude & 바이브코딩 개요

Claude란 무엇인가?

LLM (Large Language Model)

- 대규모 텍스트 데이터를 학습한 언어 모델
- 질문에 답하고, 코드를 작성하고, 문서를 생성함
- Anthropic이 개발 - 안전성과 유용성에 초점

Claude의 강점

- 긴 맥락 이해 - 코드베이스 전체를 한 번에 파악
- 코드 생성·디버깅·설명 모두 가능
- 한국어 완전 지원
- 에러 메시지를 붙여넣으면 원인과 해결책 제시

claude.ai 웹 vs Claude Code

claude.ai (웹 브라우저)

- 브라우저에서 바로 사용
- 로그인만 하면 즉시 시작
- 대화형 인터페이스
- 코드 생성 후 복사해서 VS Code에 붙여넣기
- 파일 업로드 가능 (Pro)

Claude Code (터미널)

- 터미널에서 claude 명령어로 실행
- VS Code 프로젝트 폴더와 직접 연동
- 파일 읽기/쓰기/실행 자동화
- 대형 프로젝트에 유리
- 설치 필요: npm install -g @anthropic-ai/claude-code

바이브코딩(Vibe Coding)이란?

기존 개발 방식

- 개발자가 직접 문법을 외우고 코드를 한 줄씩 작성
- 에러 → 구글링 → Stack Overflow → 수정 반복
- 진입장벽 : 언어, 프레임워크, 라이브러리를 모두 알아야 함

바이브코딩 방식

- "내가 원하는 것"을 자연어로 Claude에게 설명
- Claude가 코드를 생성 → 개발자는 실행하고 확인
- 에러 → 에러 메시지 통째로 Claude에게 → 수정 코드 받기
- 코드를 "외우는" 게 아니라 "지시하고 검토"하는 것

바이브코딩 핵심 마인드셋

"Claude는 손이고, 당신은 머리다"

무엇을 만들지 명확히 알면 된다.
코드 문법을 외우는 것이 목표가 아니다.

✔ 요구사항을 정확히 정의하는 능력
✔ Claude가 생성한 코드를 읽고 검토하는 능력
✔ 에러를 두려워하지 않는 태도 - 에러도 Claude에게 주면 된다

프롬프트: 좋은 예 vs 나쁜 예

나쁜 프롬프트

- "파이썬 코드 짜줘"
- → 무엇을? 어떤 데이터? 어떤 결과?

- "에러났어"
- → 어떤 에러? 코드가 뭔데?

- "잘 안 돼"
- → 구체적으로 어떻게 안 되는지?

좋은 프롬프트

- "CSV 파일을 읽어서 나이 컬럼의 평균을 출력하는 파이썬 코드를 작성해줘. pandas 사용."

- "아래 에러가 발생했어. 원인과 수정 코드를 알려줘 : [에러 메시지 붙여넣기]"

- "현재 코드에서 버튼 클릭 시 데이터가 저장이 안 되는데, 어디가 문제인지 봐줘"

실습: 첫 번째 Claude 코드 요청

실습 순서

- ① claude.ai 접속 및 로그인 (계정 없으면 지금 생성)
- ② 아래 프롬프트를 Claude에게 입력

  "1부터 10까지 숫자를 출력하는 파이썬 코드를 작성해줘.
  각 숫자가 홀수인지 짝수인지도 함께 출력해줘."
- ③ Claude가 생성한 코드를 VS Code에 붙여넣기
- ④ 터미널에서 python 파일명.py 실행
- ⑤ 결과 확인 후 Claude에게 "이번엔 1부터 100까지로 변경해줘" 요청

포인트

- 직접 코드를 수정하지 말고, Claude에게 말로 요청하는 연습을 할 것

VS Code 환경 구축

설치 순서 개요

Step 1. 미니콘다(Miniconda) 설치

- 다운로드 : https://docs.conda.io/en/latest/miniconda.html
  - Windows : Miniconda3 Windows 64-bit .exe 선택
  - 설치 시 'Add to PATH' 옵션 체크

Step 2. VS Code 설치

- 다운로드 : https://code.visualstudio.com
  - Python Extension 설치 (Ctrl+Shift+X → Python 검색)

Step 3. 설치 확인

```
# VS Code 터미널(Ctrl+`)에서
python --version
conda --version
```

가상환경 생성 및 패키지 설치

가상환경이란?

- 프로젝트별 독립된 Python 환경 — 패키지 충돌 방지
- Claude에게 항상 "어떤 패키지 설치해야 해?" 물어보면 됨

가상환경 생성 명령어

```
# 가상환경 생성
conda create -n vibe python=3.11

# 가상환경 활성화
conda activate vibe

# 패키지 설치 (예시)
pip install pandas requests beautifulsoup4
```

VS Code에서 가상환경 선택

- Ctrl+Shift+P → 'Python: Select Interpreter' → vibe 선택

Claude + VS Code 워크플로우

기본 작업 흐름

- ① 브라우저(claude.ai)와 VS Code를 화면 좌우로 나란히 배치
- ② Claude에게 원하는 코드 요청 → 생성된 코드 복사
- ③ VS Code에서 새 파일 생성 (예: main.py) → 붙여넣기
- ④ 터미널에서 python main.py 실행
- ⑤ 에러 발생 시 에러 메시지 전체를 Claude에 붙여넣기 → 수정

파일 저장 단축키

- Ctrl+S : 저장 | Ctrl+` : 터미널 열기 | Ctrl+Z : 되돌리기

실습: 환경 세팅 완료 확인

실습 체크리스트

- ☐ 미니콘다 설치 완료 → conda --version 출력 확인
- ☐ VS Code 설치 완료 + Python Extension 활성화
- ☐ 가상환경 'vibe' 생성 및 활성화
- ☐ pandas, requests 패키지 설치

최종 확인 코드

```
# Claude에게 요청: "pandas가 설치됐는지 확인하는 코드 작성해줘"
import pandas as pd
print('pandas 버전:', pd.__version__)
print('환경 세팅 완료!')
```

- → 위 코드 실행 후 버전 출력되면 준비 완료

바이브코딩용 Python 기초

핵심 원칙: 읽기 수준으로만

외울 필요 없는 것들

- Python 문법 규칙을 암기하는 것
- 라이브러리 함수명을 기억하는 것
- 처음부터 코드를 짜는 것

꼭 알아야 하는 것들

- 변수에 값이 저장된다는 개념
- 리스트(여러 값)와 딕셔너리(키-값 쌍)의 구조
- 함수가 "입력을 받아 처리 후 출력한다"는 개념
- if/for 블록이 무엇을 제어하는지

바이브코딩의 현실

- 코드를 못 짜도 된다. Claude가 생성한 코드를 검토하는 눈이 필요하다.

변수와 기본 자료형

변수 — 값을 담는 그릇

```
name = "홍길동"        # 문자열 (str)
age = 30               # 정수 (int)
score = 98.5           # 실수 (float)
is_active = True       # 논리값 (bool)
```

읽는 법

- = 는 '같다'가 아니라 '오른쪽 값을 왼쪽에 저장'
- # 이후는 주석 — 코드 설명, 실행에 영향 없음
- print(name) 하면 "홍길동" 출력

Claude 활용

- "name, age, score 변수를 사용해서 자기소개 문장을 출력하는 코드 작성해줘"

리스트와 딕셔너리

리스트 — 순서가 있는 여러 값

```
fruits = ["사과", "바나나", "오렌지"]
print(fruits[0])   # → 사과 (0부터 시작)
```

딕셔너리 — 키-값 쌍

```
student = {
    "name": "홍길동",
    "age": 30,
    "score": 98.5
}
print(student["name"])   # → 홍길동
```

언제 쓰나?

- 리스트 : 같은 종류의 데이터 여러 개 (학생 목록, 점수 목록)
- 딕셔너리 : 한 대상의 여러 속성 (학생 정보, 상품 정보)

함수, 조건문, 반복문

함수 — 반복 작업을 묶어놓은 것

```
def greet(name):        # name을 입력받아
    return f"안녕, {name}!"  # 문자열 반환

print(greet("홍길동"))  # → 안녕, 홍길동!
```

조건문

```
if score >= 90:
    print("A학점")
else:
    print("B학점 이하")
```

반복문

```
for fruit in fruits:
    print(fruit)   # 리스트의 각 항목 순서대로 출력
```

Claude 생성 코드 해석 실습

실습 방법

- ① Claude에게 코드 생성 요청

  "학생 이름과 점수가 담긴 딕셔너리 리스트를 만들고
  평균 점수를 계산해서 출력하는 코드 작성해줘"

- ② 생성된 코드를 보고 각 줄이 무엇을 하는지 설명해보기
- ③ Claude에게 "이 코드에서 변수가 몇 개야? 각각 설명해줘" 요청

에러 대처법

- 에러 발생 → 터미널의 빨간 텍스트 전체 복사
- Claude에게 : "아래 에러가 발생했어. 원인과 수정 코드 알려줘 : [붙여넣기]"

프롬프트 전략 & 개발 워크플로우

좋은 프롬프트의 4가지 요소

① 목적 (무엇을 만드나?)

- "CSV 파일을 읽어서 특정 컬럼의 평균을 계산하는 코드"

② 환경 (어떤 도구를 쓰나?)

- "Python, pandas 라이브러리 사용"

③ 데이터 형태 (어떤 입력인가?)

- "컬럼: name, age, score가 있는 CSV"

④ 출력 형태 (결과를 어떻게 보여주나?)

- "결과를 터미널에 "평균 점수: XX.X" 형식으로 출력"

완성 예시

```
"name, age, score 컬럼이 있는 CSV를 pandas로 읽어서
 score의 평균을 '평균 점수: XX.X' 형식으로 출력하는 코드 작성해줘"
```

단계별 요청 vs 한번에 요청

한번에 모두 요청

- "CSV 읽어서 데이터 정제하고 평균 계산하고 그래프도 그리고 결과를 엑셀로 저장하는 코드 만들어줘"

- → 코드가 너무 길어져 오류 발생 가능
- → 어디서 문제가 생겼는지 파악 어려움
- → 수정 요청이 복잡해짐

단계별로 쪼개서 요청

- Step 1: "CSV 파일 읽는 코드만 작성해줘"
- → 실행 확인

- Step 2: "이제 NaN 값 제거하는 코드 추가해줘"
- → 실행 확인

- Step 3: "score 컬럼 평균 계산 추가해줘"
- → 실행 확인

- Step 4: "matplotlib로 막대그래프 추가해줘"

디버깅 요청 패턴

에러 발생 시 Claude에게 전달할 정보

- ① 현재 코드 전체 (또는 에러 발생 부분)
- ② 에러 메시지 전체 (터미널 빨간 텍스트 전부)
- ③ 기대했던 동작 vs 실제 동작

좋은 디버깅 프롬프트 예시

```
"아래 코드를 실행했더니 에러가 발생했어.
 기대: CSV 파일을 읽어서 평균 출력
 실제 에러: FileNotFoundError: data.csv 없음

 [코드 붙여넣기]
 [에러 메시지 붙여넣기]

 원인과 수정 방법 알려줘"
```

실습: 프롬프트 개선

실습 내용

- 아래의 나쁜 프롬프트를 4가지 요소를 활용해 개선해보기

나쁜 프롬프트 (개선 대상)

1. "데이터 분석 코드 짜줘"
2. "파일 저장해줘"
3. "에러났어"

진행 방법

- ① 각자 개선된 프롬프트 작성 (3분)
- ② 작성한 프롬프트를 Claude에 입력 → 결과 비교
- ③ 나쁜 프롬프트 결과 vs 좋은 프롬프트 결과 차이 확인

FastAPI 바이브코딩

API란 무엇인가?

API (Application Programming Interface)

- 프로그램끼리 대화하는 창구
- 클라이언트(요청) ↔ 서버(응답) 구조
- 예: 날씨 앱이 기상청 서버에 날씨 데이터를 요청하는 방식

REST API 기본 개념

- GET: 데이터 조회 (읽기)
- POST: 데이터 생성 (쓰기)
- PUT: 데이터 수정
- DELETE: 데이터 삭제

FastAPI

- Python으로 API를 빠르게 만들 수 있는 프레임워크
- 자동으로 Swagger UI(테스트 페이지) 제공

FastAPI 설치 및 첫 실행

패키지 설치

```
pip install fastapi uvicorn
```

Claude에게 요청할 프롬프트

```
"FastAPI로 GET /hello 엔드포인트를 만들어줘.
 요청하면 {"message": "안녕하세요!"} 를 반환해.
 uvicorn으로 실행하는 방법도 알려줘."
```

실행 및 확인

```
uvicorn main:app --reload
```

- 브라우저에서 http://localhost:8000/docs 접속
- Swagger UI에서 GET /hello 직접 테스트

실습: 학생 정보 CRUD API

실습 목표: 학생 정보를 관리하는 API 만들기

Step 1 — 학생 목록 조회 API

```
"GET /students 엔드포인트를 만들어줘.
 학생 목록(이름, 나이, 점수)을 리스트로 반환해.
 초기 데이터 3명분을 하드코딩으로 넣어줘."
```

Step 2 — 학생 추가 API

```
"POST /students 엔드포인트를 추가해줘.
 name, age, score를 받아서 목록에 추가해."
```

Step 3 — Swagger에서 테스트

- GET /students → 학생 목록 확인
- POST /students → 새 학생 추가 후 다시 GET으로 확인

에러 대처 & 마무리

자주 발생하는 에러

- ModuleNotFoundError → pip install 해당패키지
- Address already in use → 포트 변경: --port 8001
- 422 Unprocessable Entity → 요청 데이터 형식 문제

모든 에러 → Claude에게 전달

```
"아래 에러가 발생했어. 원인과 수정 코드 알려줘:
 [에러 메시지 전체 붙여넣기]"
```

핵심 정리

- FastAPI 코드를 외울 필요 없다
- 구조(엔드포인트, 요청/응답)를 이해하고 Claude에게 지시

Streamlit 바이브코딩

Claude로 웹 UI를 만드는 실전 실습

FastAPI vs Streamlit — 역할 구분

FastAPI (백엔드)

- 데이터를 처리하고 저장하는 서버
- 브라우저에서 바로 보이지 않음
- API 엔드포인트로 데이터 제공
- 예: 학생 정보 저장/조회 로직

- → '주방' 역할
- (손님은 직접 못 들어감)

Streamlit (프론트엔드)

- 사용자가 직접 보고 사용하는 화면
- 브라우저에서 바로 실행
- FastAPI에 요청해서 데이터 표시
- 예: 학생 정보 입력폼, 목록 화면

- → '홀' 역할
- (손님이 직접 사용하는 공간)

Streamlit 설치 및 첫 실행

패키지 설치

```
pip install streamlit requests
```

Claude에게 요청할 프롬프트

```
"Streamlit으로 간단한 웹앱을 만들어줘.
 - 제목: '학생 관리 시스템'
 - 텍스트 입력: 이름, 나이, 점수
 - 버튼: '추가'
 - 버튼 클릭 시 입력값을 화면에 출력"
```

실행 및 확인

```
streamlit run app.py
```

- 브라우저가 자동으로 열림: http://localhost:8501

실습: FastAPI + Streamlit 연동

Step 1 — M5에서 만든 FastAPI 서버 실행

```
# 터미널 1
uvicorn main:app --reload
```

Step 2 — Streamlit에서 API 연동

```
"Streamlit 앱에서 GET http://localhost:8000/students
 를 호출해서 학생 목록을 표로 보여주는 코드 작성해줘.
 requests 라이브러리 사용."
```

Step 3 — 학생 추가 폼 연동

```
"이름/나이/점수 입력 후 '추가' 버튼 클릭 시
 POST /students 를 호출하는 코드 추가해줘."
```

Step 4 — 터미널 2에서 Streamlit 실행

```
# 터미널 2
streamlit run app.py
```

UI 개선: 차트 추가

데이터 시각화 추가 요청

```
"학생 목록에서 score 컬럼으로 막대그래프를 그려줘.
 x축: 학생 이름, y축: 점수.
 streamlit의 st.bar_chart 사용."
```

추가 UI 컴포넌트 요청 예시

- "점수 70 미만인 학생은 빨간색으로 표시해줘"
- "학생 목록을 점수 내림차순으로 정렬해줘"
- "사이드바에 필터(나이 범위 슬라이더) 추가해줘"

핵심

- 원하는 UI를 말로 설명하면 Claude가 컴포넌트 코드를 생성한다.
- 세부 스타일도 자연어로 요청하면 됨.

강의 마무리

**"요구사항을 정확히 말하는 능력이
개발 실력이다"**

- ✔ 코드를 외우는 것이 목표가 아니다
- ✔ Claude에게 단계별로, 구체적으로 요청하는 것이 핵심
- ✔ 에러를 두려워하지 말 것 — 에러도 Claude에게 주면 된다
- ✔ 오늘 배운 패턴(요청 → 실행 → 수정)이 실무에서도 그대로 통한다

수고하셨습니다

이메일 문의
gilsikpark@kopo.ac.kr
gspark@ewha.ac.kr